

Project Overview

- FitLens is a local fitness analytics CLI that combines Hevy workout exports with Apple Health data.
- It builds a local SQLite database and turns raw exports into workout, recovery, and coaching insights.

What Is Implemented So Far

- Apple Health XML parsing
- Hevy workout CSV parsing
- SQLite database import pipeline
- Guided CLI flow using rich and questionnaire
- Recent workouts view
- Weekly training load view
- Recovery windows: 7, 30, 60, and 90 day averages
- Rule-based coach insights and next-month recommendations
- Movement balance / muscle group mapping
- Synthetic demo data for safe demonstration

Current Demo

1. First run, post onboarding of 2 files, menu select screen:

```
o ((.venv) ) jamespattison@Jamess-Mac-mini FitLens % python3 fitlens.py
FitLens

Welcome back to FitLens 🙌

You currently have 195 workouts, 339 days of health data, and 229 nights of sleep.
Training spans 2025-06-15 → 2026-05-15 · last import 2026-06-15

? What do you want to do? (Use arrow keys)
> Coach insights
  Coach recommendations
  Movement balance
  Recent workouts
  Weekly training summary
  Recovery trends
  Database status
  Import new data
  Quit
```

2. Coach Insights:

```
? What do you want to do? Coach insights

Data window: May 11 – May 17, 2026
Baseline comparison: Apr 18 – May 17, 2026

Training (May 11 – May 17, 2026)
Workouts: 5 (3.3h)
Prior 7 days: 3 (2.2h)
Avg workout effort: 3.3

Recovery context
Sleep: 7.9h avg vs 7.1h baseline
HRV: 73.7 avg vs 72.9 baseline
Resting HR: 60.6 avg vs 59.3 baseline

Most recent workout
2026-05-15 4x4 HIIT & Abs – 16m, avg HR 128, effort 2.9

Coach notes
• Training time is up 70 minutes versus the prior 7 days.
• Average sleep is 7.9 hours over the latest 7 days, which supports harder training.

? What do you want to do? (Use arrow keys)
> Coach insights
Coach recommendations
Movement balance
Recent workouts
Weekly training summary
Recovery trends
Database status
Import new data
Quit
```

This screen is meant to be the quick “what is going on right now?” view. In the code, the CLI calls `render_insights()` in `fitlens.py`, which gets its data from `insights.coach_insights(db_path)`. That function pulls recent workout totals, recovery metrics, and the most recent workout from the SQLite database.

For example, it compares my current 7-day training window against the previous 7 days, then adds recovery context like sleep, HRV, and resting heart rate. I wanted this screen to feel like a coach summary instead of just a raw table of numbers.

3. Coach recommendations:

```
? What do you want to do? Coach recommendations

Readiness: GREEN
Current month: Apr 18 – May 17, 2026
Compared with: Mar 19 – Apr 17, 2026

Trainings: 19 workouts, 11.9h (~7.3%)
Strength: 210 sets (+22.1%)
Cardio: 4.1h total, 3.2h Zone 2, 54m hard
Sleep: 7.1h avg

recovery and training load look stable

Priority Actions



| Priority | Area             | Recommendation                                                          | Action                                                                                   | Timeframe    |
|----------|------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------|
| High     | Strength         | Push lifting volume slightly                                            | Add 5–10% more work by adding 1 set to a few main lifts, not every lift.                 | Weeks 2–4    |
| Medium   | Cardio           | Readiness is green with 210 strength sets in the latest 30 days.        | Add one easy aerobic session weekly before adding more hard intervals.                   | Weeks 1–2    |
| Medium   | Strength Balance | Bring Zone 2 back up                                                    | Add 4–6 sets of rows or pulldowns each week to even chest and back.                      | Next 4 weeks |
| Medium   | Strength Balance | Add back volume                                                         | Add 4–6 sets of face pulls or reverse flys each week.                                    | Next 4 weeks |
| Low      | Recovery         | Back (40 sets) trails chest (57) this month.                            | Leave at least one easy or rest day between intervals and your hardest lifting sessions. | Next 4 weeks |
| Low      | Strength Balance | Bring up rear delts                                                     | Add 3–4 sets of hip-hinge (RDL/deadlift) weekly so both functions get trained.           | Next 4 weeks |
|          |                  | Rear delts (6 sets) lag front/side (40).                                |                                                                                          |              |
|          |                  | Keep hard days separated                                                |                                                                                          |              |
|          |                  | Hard cardio is about 54 min in the latest 30 days.                      |                                                                                          |              |
|          |                  | Hit both hamstring functions                                            |                                                                                          |              |
|          |                  | Hamstring work is all one pattern; hip-hinge (RDL/deadlift) is missing. |                                                                                          |              |



Next 4 Weeks



| Week   | Lifting             | Cardio           | Intensity                | Progression                                            |
|--------|---------------------|------------------|--------------------------|--------------------------------------------------------|
| Week 1 | 52–57 strength sets | 50–80 Zone 2 min | 0–1 hard cardio sessions | Hold this month's rhythm and keep hard days separated. |
| Week 2 | 52–57 strength sets | 50–80 Zone 2 min | 0–1 hard cardio sessions | Add one small lift progression or 10–15 min Zone 2.    |
| Week 3 | 52–57 strength sets | 50–80 Zone 2 min | 0–1 hard cardio sessions | Progress another 5% if sleep and HRV stay steady.      |
| Week 4 | 52–57 strength sets | 50–80 Zone 2 min | 0–1 hard cardio sessions | Keep volume near week 3, then review the next export.  |


```

This is probably the most important Version 1 feature. The code calls `insights.coach_recommendations(db_path)`, which compares the latest 30 days against the previous

30 days. It calculates things like workout duration, strength sets, cardio minutes, Zone 2 minutes, hard cardio minutes, sleep, HRV, and resting heart rate.

The recommendations are rule-based right now. For example, if training load jumps too much, sleep is low, HRV drops, or resting heart rate increases, the readiness score can move from green to yellow or red. Then the app creates priority actions and a basic four-week plan. This is not AI yet, but it proves the data pipeline works and that I can generate useful feedback from the imported data.

4. Movement Balance:

```
? What do you want to do? Movement balance Movement Balance

Window: Apr 18 - May 17, 2026 (210 sets, 100% classified)
Fractional sets: 1.0 per primary muscle, 0.5 per assisting muscle.

Volume by muscle group
Chest 57.0 (upper 18.0, mid 39.0)
Shoulders 49.0 (front 22.5, side 18.0, rear 8.5)
Back 39.5 (lats 22.5, rows 17.0)
Triceps 38.5
Biceps 32.5
Quads 24.0 (compound 15.0, isolation 9.0)
Hamstrings 16.0 (curl 16.0)
Glutes 25.5
Adductors 7.5
Abductors 0.0
Calves 0.0
Core 13.0 (abs 7.0, obliques 6.0)

Balance (same-role muscles, not push vs pull)
Chest : Back 57 : 40 (1.44x)
Quads : Hams 24 : 16 (1.50x)
Biceps : Triceps 32 : 38 (0.84x)
```

This screen uses the exercise taxonomy I wrote in `taxonomy.py`. The basic idea is that each exercise gets classified into muscle groups. Some exercises are exact matches, and others use regex fallback matching. Then `insights.movement_balance()` counts the sets from the latest 30 days and turns them into muscle volume.

For example, an incline press does not only count as “chest.” It mainly counts toward upper chest, but it can also partially count toward front delts and triceps. That is why I used fractional volume instead of just one muscle per exercise (A recent meta-analysis shows fractional sets for secondary muscles align the closest with muscle growth outcomes). This lets the app show ratios like chest vs. back or quads vs. hamstrings, which is more useful for spotting imbalances.

5. Recent Workouts:

? What do you want to do? **Recent workouts**
Recent Workouts (10)

Date	Time	Workout	Min	Sets	Volume	Avg HR	Effort
2026-05-15	19:26	4x4 HIIT & Abs	16	8	4,095	128	2.9
2026-05-14	19:28	Posterior FB	48	17	12,235	131	2.6
2026-05-13	19:00	Zone2 cardio	60	1	0	145	5.0
2026-05-13	18:42	Zone2 cardio	16	8	5,925	117	3.0
2026-05-12	20:14	Anterior FB	60	20	27,505	114	3.1
2026-05-08	13:06	Full Body	62	20	31,715	111	2.8
2026-05-06	20:29	4x4 HIIT & Abs	22	8	2,160	138	n/a
2026-05-04	20:00	Posterior FB	46	16	12,430	115	2.7
2026-05-03	19:50	Anterior FB	24	10	10,825	122	3.1
2026-05-03	19:36	Full Body	14	7	13,150	108	1.8

? What do you want to do? (Use arrow keys)
» Coach insights

This screen is built by `render_recent()` in `fitlens.py`, using data from `insights.recent_workouts()`. It queries the local SQLite database and joins workout data with health summaries, so I can show both workout information and Apple Health-style metrics in one table.

For example, the table can show the workout title, duration, number of sets, volume, average heart rate, and effort. This is useful because it proves that the Hevy workout export and Apple Health data are being connected correctly instead of just imported separately.

6. Weekly Training Summary

? What do you want to do? **Weekly training summary**
Weekly Training Load (8 weeks)

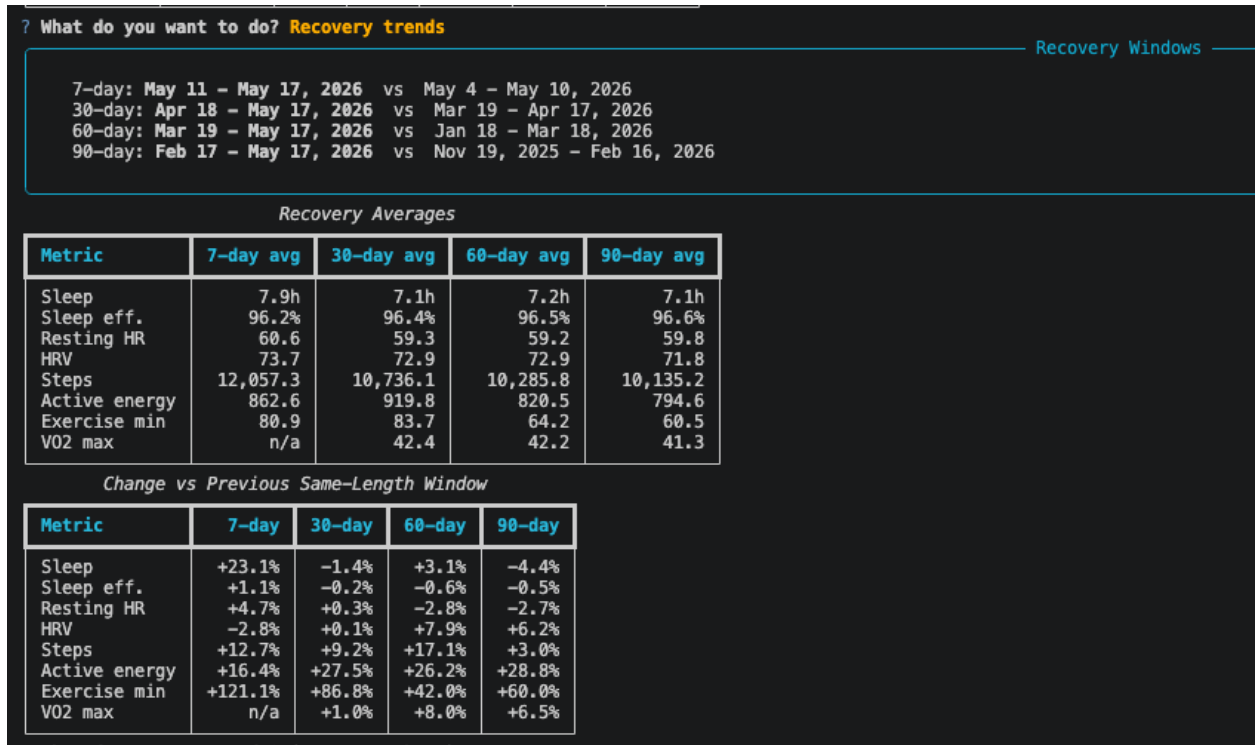
Week Of	Workouts	Time	Sets	Volume	Avg HR	Effort
2026-05-11	5	3.3h	54	49,760	127	3.3
2026-05-04	3	2.2h	44	46,305	121	2.7
2026-04-27	4	2.0h	45	62,230	117	2.8
2026-04-20	5	3.3h	56	50,095	132	3.3
2026-04-13	2	64m	25	33,935	115	1.8
2026-04-06	5	3.3h	47	48,265	129	3.3
2026-03-30	4	2.5h	34	30,895	128	3.7
2026-03-23	7	5.1h	65	71,505	133	3.8

? What do you want to do? (Use arrow keys)
» Coach insights

This view is generated by `render_weekly()` and the query logic in `insights.weekly_summary()`. Instead of looking at one workout at a time, it groups workouts by week and calculates totals like workout count, duration, sets, volume, average heart rate, and average effort.

Technically, this is one of the views that makes SQLite useful. Once the import pipeline stores everything in tables, I can write queries that group and summarize the data in different ways. This helped me move from “I imported my data” to “I can actually analyze trends.”

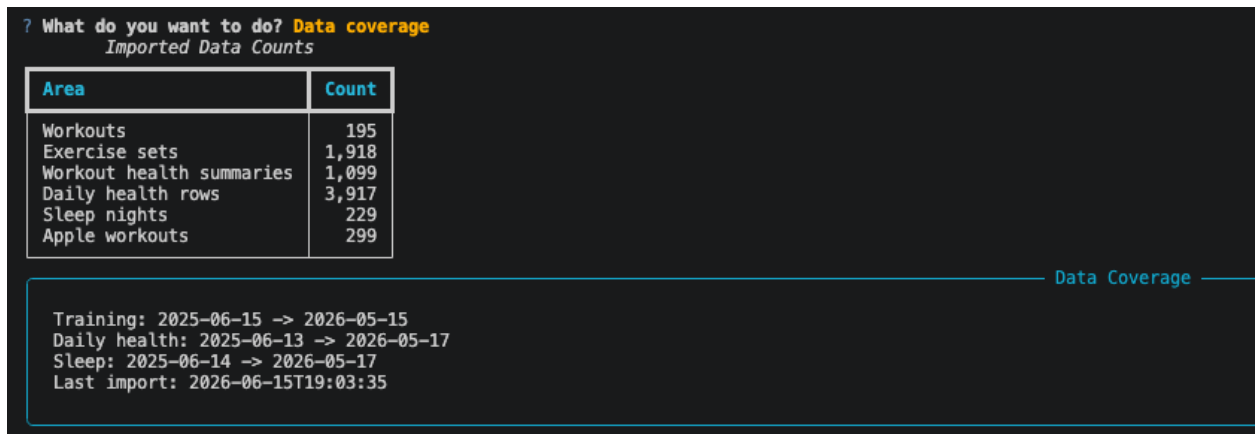
7. Recovery Trends



This screen comes from `render_recovery()` in `fitlens.py` and `insights.recovery_summary()`. The code looks at the newest date in the imported data, then builds 7-day, 30-day, 60-day, and 90-day windows based on that date. I did that on purpose because my latest export might not end on today's date.

The recovery metrics come from Apple Health data stored in `daily_health` and `daily_sleep`. The app calculates averages for sleep, sleep efficiency, resting heart rate, HRV, steps, active energy, exercise minutes, and VO2 max. It also compares each window to the previous same-length window, so the 30-day average is compared against the prior 30 days, not just some random baseline.

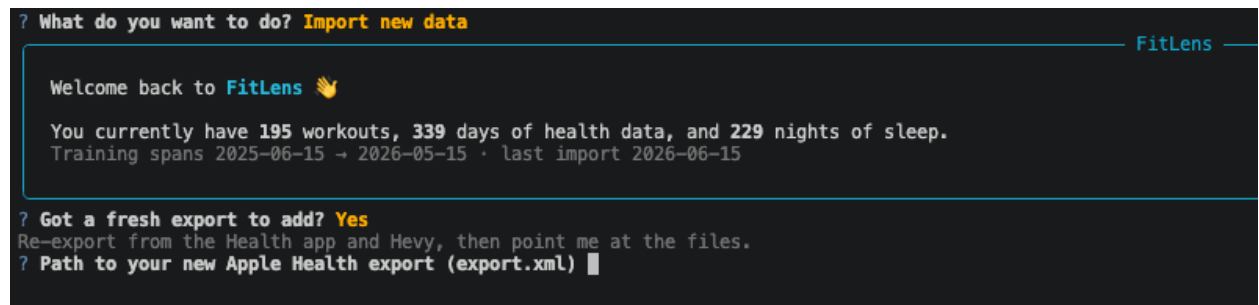
8. Data Coverage



This screen used to be called “Database status,” but I renamed it because “Data coverage” better describes what it actually does. In the code, it is still handled by `render_status()`, but the user-facing label is now clearer.

This screen pulls counts from the main SQLite tables, like `workouts`, `workout_sets`, `workout_health_summary`, `daily_health`, `daily_sleep`, and `apple_workouts`. It also shows date ranges for training data, daily health data, and sleep data. I use this screen to check whether the import worked and whether FitLens has enough data to make the other screens meaningful.

9. Import New Data



The import flow is handled mostly in `engine.py`. The main function is `engine.ingest()`, which takes the Apple Health XML file, the Hevy CSV file, the database path, timezone, and lookback settings. It parses the Hevy CSV with `parse_workouts.py`, streams the Apple Health XML with `health_stream.py`, and writes everything into SQLite through `db.py`.

One technical thing I had to think about was not re-importing everything every time. The app stores metadata like `last_ingested_ts`, `coverage_start`, file paths, and timezone in the `import_meta` table. It also re-reads a small overlap window so the latest day of health/sleep data can settle correctly. That makes the import process more practical for repeated use instead of only working once.

Issues Encountered

- Apple Health exports are large, so streaming/parsing was needed instead of loading everything casually.
- Matching workouts to health metrics required careful time-window logic.
- A pure command-flag CLI was awkward (not very user-friendly IMO), so the app shifted to a guided menu (much better!).
- Muscle/exercise classification needed a taxonomy because exercise names vary. Had to use AI as my rubber ducky to think through the whole taxonomy and regex to make it encompass as much as possible.
- Python 3.14 caused prompt UI import hangs, so the project uses Python 3.12.
- This issue was specific to my setup but I built my project in my Mac’s Documents folder which is iCloud synced. This caused me many long hours of issues with the python’s virtual environment. Finally moved it to a local folder and everything was fixed.

Unsolved Issues / Help Needed

- No polished GUI yet; app is CLI-first.
- Recommendations are rule-based, not AI-generated. Would like a nice mix as some deterministic heuristics are still good.
- Exercise taxonomy may need expansion as more exports are tested and a lightweight LLM parser could help too.
- Possible help request: feedback on whether Version 2 should prioritize a simple UI, smarter coaching, or better data validation.

Version 2 Milestones

- **Milestone 1: Submission Polish**
Create review/spec docs, add screenshots, clean repo structure.
- **Milestone 2: Reliable Import Experience**
Improve error messages, file validation, and re-import behavior.
- **Milestone 3: Better Coaching Layer**
Expand rule-based recommendations and make explanations more transparent.
- **Milestone 4: Visualization Layer**
Add charts for training load, recovery, and muscle balance.
- **Milestone 5: Usability Upgrade**
Build a beautiful dashboard interface for better user experience.

Self Reflection

- Progress is solid because the core data pipeline works end to end.
- The project is finishable if Version 2 focuses on polish and presentation rather than over-expanding. I have never created a desktop application with Tkinter but, from the examples ive looked up online, it seems very user-friendly and straight forward.
- Easier than expected: basic CLI workflow and SQLite storage. Prior 6 years of software engineering experience definitely helped me speed through. Having to look up python specific syntax was the longest part but only resources are extensive.
- Harder than expected: Apple Health data shape (still think its terrible), workout matching, useful recommendation logic (I have coached multiple people on the side so I want to make sure recommendation logic is rooted in realistic, actionable and evidence-based recommendations).
- Light bulb moment: the hard part is not just importing data, but turning it into trustworthy advice.